



Expressions in C

Understanding Type Conversion and Evaluation



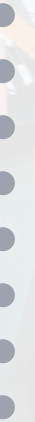
Operators



Type Conversion



Casting





What are Expressions?



An expression is a **combination of variables, constants, and operators** arranged according to the syntax of the language.

Examples of Expressions:

```
c = (m + n) * (a - b);
```

```
temp = (a + b + 'c') / (d - c);
```

Evaluation Process:

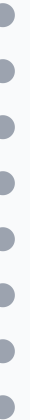


Expression is evaluated using assignment statement of the form:

```
Variable = expression;
```



All relevant variables must be assigned values before evaluation!





Type Conversion in Expressions



Type conversion refers to the process where values of one data type are **transformed into another** to ensure compatibility within operations.

Implicit Conversion

Automatically handled by the programming language

Occurs in expressions with mixed data types

Explicit Conversion

Manually converted by the programmer

Also called type casting



Why Important: Both types of conversions are crucial for writing robust and error-free code, ensuring that expressions evaluate as intended.





Automatic Type Conversion



If operands are different types, the **lower type is automatically converted** to the higher type before the operation proceeds.

Conversion Rules:

Op-1	Op-2	Result
any	long double	long double
any	double	double
any	float	float
any	unsigned long int	unsigned long int
any	long int	long int
any	unsigned int	unsigned int





Note: The final result of an expression is converted to the type of the variable on the left of the assignment sign before assigning value to it.



Final Assignment Rules



The following changes are introduced during the final assignment:



float to int

Causes truncation of the fractional part

```
float x = 3.99;  
int y = x; // y will be 3, not 4
```



double to float

Causes rounding of digits

```
double x = 3.14159265359;  
float y = x; // y will be 3.141593
```





long int to int

Causes dropping of the excess higher order bits

```
long int x = 1234567890;  
int y = x; // y will lose some bits
```



Casting a Value



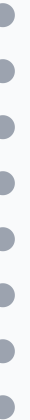
Casting a value is **forcing a type conversion** in a way that is different from the auto conversion. This process is called **type cast**.

General Form of Casting:

```
(type_desired) expression;
```

type_desired: standard C data types

expression: constant, variable or expression





Why Use Casting? Type casting is necessary when automatic conversions do not yield the desired outcome or when precision and control over the data transformation are required.



Type Casting Example

```
#include<stdio.h>
void main() {
    int total_marks = 500, ob_marks = 234;
    float per1, per2;

    per1 = (ob_marks / total_marks) * 100;
    per2 = (float)(ob_marks / total_marks) * 100;

    printf("Percentage without type casting = %.2f", per1);
    printf("Percentage After type casting = %.2f", per2);
    getch();
}
```

Output:

Percentage without type casting = 0.00

Percentage After type casting = 46.80



Explanation: In the expression `per2 = (float)(ob_marks / total_marks) * 100;`, the division is converted to float, otherwise the decimal part of the result would be lost and `per1` would represent a wrong figure or zero.



Conclusion



Operators

Discussed different types of operators: arithmetic, relational, logical



Type Conversions

Understanding conversions prevents unexpected results and logical errors



C Shorthand



Conditional Operator

C is a compact language with shorthand expressions

Short form of writing the if/else construct



These concepts are fundamental and will be used in control statements, arrays, and other C constructs!

